

形式语言与自动机

Formal Languages and Automata Theory

正则表达式

计算机科学与技术学院
哈尔滨工业大学（深圳）

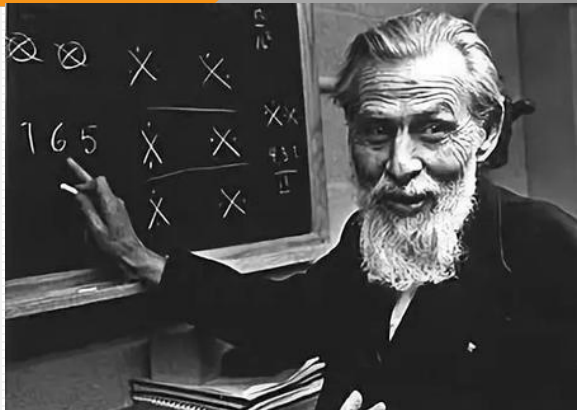


郑宜峰

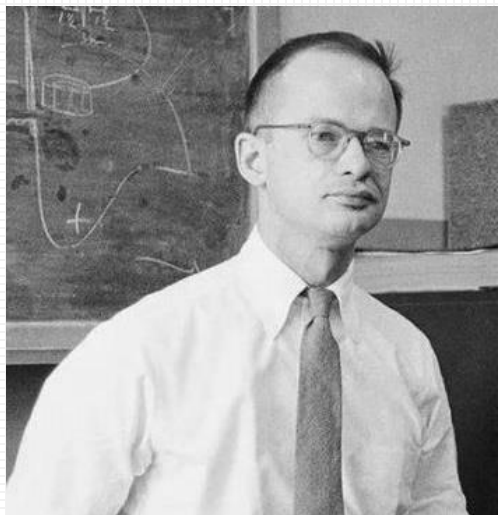
- **正则表达式**
 - 正则表达式的递归定义
 - 正则表达式的示例
- **有穷自动机和正则表达式**
 - 由自动机到正则表达式
 - 由正则表达式到自动机
- **正则表达式的代数定律**

- **正则表达式**
 - 正则表达式的递归定义
 - 正则表达式的示例
- 有穷自动机和正则表达式
 - 由自动机到正则表达式
 - 由正则表达式到自动机
- 正则表达式的代数定律

正则表达式的起源



沃伦·麦卡洛克
神经学家，计算神经科学的开创者之一



沃尔特·皮茨
数学家，与麦卡洛克一起研究出世界上第一个神经元网络模型



斯蒂芬·科尔·克莱因
数学家，他的递归论研究有助于奠定理论计算机科学的基础

- 正则表达式的“鼻祖”或许可一直追溯到科学家对人类神经系统工作原理的早期研究。美国新泽西州的Warren McCulloch和出生在美国底特律的Walter Pitts这两位神经生理方面的科学家，研究出了一种用数学方式来描述神经网络的新方法。
- 在1951年，一位名叫Stephen Kleene的数学科学家，他在Warren McCulloch和Walter Pitts早期工作的基础之上，发表了一篇题目为《神经网络与有穷自动机中的事件表示》（Representation of Events in Nerve Nets and Finite Automata）的论文，引入了正则表达式的概念。

正则表达式的发展



肯·汤普森(Ken Thompson)

1943~

美国计算机科学学者
C语言前身B语言的作者
Unix发明人之一



Linux



Microsoft

- 之后一段时间，人们发现可以将正则表达式应用于其他方面。**Ken Thompson**就把这一成果应用于搜索算法的一些早期研究，Ken Thompson是 Unix 的主要发明人，也就是大名鼎鼎的**Unix之父**。自此以后，正则表达式被广泛地应用到各种UNIX或类似于UNIX的工具中，如大家熟知的**Perl**。
- 不仅众多UNIX工具支持正则表达式，近二十年来，在WINDOWS的阵营下，正则表达式的思想和应用在大部分 Windows 开发者工具包中得到支持和嵌入应用。

- 主流操作系统（*nix[Linux, Unix等]、Windows、HP、BeOS等）、主流的开发语言（Delphi、Scala、PHP、C#、Java、C++、Objective-c、Swift、VB、Javascript、Ruby以及Python等）、数以亿万计的各种应用软件中，都可以看到正则表达式的使用。

- **有穷自动机**
 - 通过抽象机器装置扫描和接受字符串识别（表示）正则语言
- **正则表达式**
 - 通过代数式(algebraic expression)表示正则语言
 - 正则表达式所表示的语言与正则语言等价
 - 应用广泛，比如grep工具、程序设计语言（如 Perl, Python）

- 设 L 和 M 是两个语言, 那么

并: $L \cup M = \{w | w \in L \text{ 或 } w \in M\}$

连接: $L \cdot M = \{w | w = xy, x \in L \text{ 且 } y \in M\}$

幂: $L^0 = \{\epsilon\}$, $L^1 = L$, $L^n = L^{n-1} \cdot L$,

克林闭包: $L^* = \bigcup_{i=0}^{\infty} L^i$

- 例:若有 $L = \{0,11\}$, $M = \{\varepsilon, 001\}$, 那么
 - $L \cup M = \{0,11,\varepsilon, 001\}$
 - $L \cdot M = \{0,0001,11,11001\}$
 - $M \cdot L = \{0,11,0010,00111\}$
 - $L^0 = \{\varepsilon\}, L^1 = L, L^2 = \{00,011,110,1111\}$
- 例:对于空语言 $\emptyset$
 - $\emptyset^0 = \{\varepsilon\}, \emptyset^n = \emptyset, \emptyset^* = \{\varepsilon\}$

回顾: 集合 A 的 n 次幂 ($n \geq 0$), 递归定义为

$$A^n = \begin{cases} \{\varepsilon\}, & n = 0 \\ A^{n-1}A, & n \geq 1 \end{cases}$$

- 任何数都是四则运算表达式。
- 如果 a 和 b 是四则运算表达式，那么

$$a + b, a - b, a \times b, a \div b, (a)$$

都是四则运算表达式。

- 基础1: $\emptyset$ 是一个正则表达式, 表示空语言 $\emptyset$ 。
- 基础2: ε 是一个正则表达式, 表示语言 $\{\varepsilon\}$ 。
- 基础3: 对于任意一个符号 a , a 是一个正则表达式, 表示语言 $\{a\}$, 其有一个长度为1的字符串。

- 归纳1：如果 E_1 和 E_2 是正则表达式，那么 $E_1 + E_2$ 也是正则表达式，且 $L(E_1 + E_2) = L(E_1) \cup L(E_2)$ 。
- 归纳2：如果 E_1 和 E_2 是正则表达式，那么 $E_1 E_2$ 也是正则表达式，且 $L(E_1 E_2) = L(E_1) L(E_2)$ 。
- 归纳3：如果 E 是正则表达式，则 E^* 也是正则表达式。
- 归纳4：如果 E 是正则表达式，则 (E) 也是正则表达式，表示语言 $L(E)$ 。

- 正则表达式中三种运算以及括号的优先级：

(1) “括号” 优先级最高

(2) “星” 运算: E^*

(3) “连接” 运算: E_1E_2

(4) “加” 最低: $E_1 + E_2$

括号 ()		高
“星” *		
“连接” (·一般省略)		
“加” +		低

- 例: $1 + 01^* = 1 + (0(1^*))$
 - 优先级: $() > * > \cdot > +$

所表示的语言集合为:

- 例: $1 + 01^* = 1 + (0(1^*))$
 - 优先级: $() > * > \cdot > +$

所表示的语言集合为:

$$(1^*) = \{\varepsilon, 1, 11, 111, 1111, \dots\}$$

- 例: $1 + 01^* = 1 + (0(1^*))$
 - 优先级: $() > * > \cdot > +$

所表示的语言集合为:

$$(1^*) = \{\varepsilon, 1, 11, 111, 1111, \dots\}$$
$$0(1^*) = \{0, 01, 011, 0111, \dots\}$$

- 例: $1 + 01^* = 1 + (0(1^*))$
 - 优先级: $() > * > \cdot > +$

所表示的语言集合为:

$$(1^*) = \{\varepsilon, 1, 11, 111, 1111, \dots\}$$

$$0(1^*) = \{0, 01, 011, 0111, \dots\}$$

$$1 + (0(1^*)) = \{1, 0, 01, 011, 0111, \dots\}$$

- 给出正则表达式 $(a+b)^*(a+bb)$ 定义的语言

优先级: $() > * > \cdot > +$

- 给出正则表达式 $(a+b)^*(a+bb)$ 定义的语言

优先级: $() > * > \cdot > +$

- $a \rightarrow \{a\}, b \rightarrow \{b\}$

- $a + b = \{a\} \cup \{b\} = \{a, b\}, (a + b)^* = \{a, b\}^*$

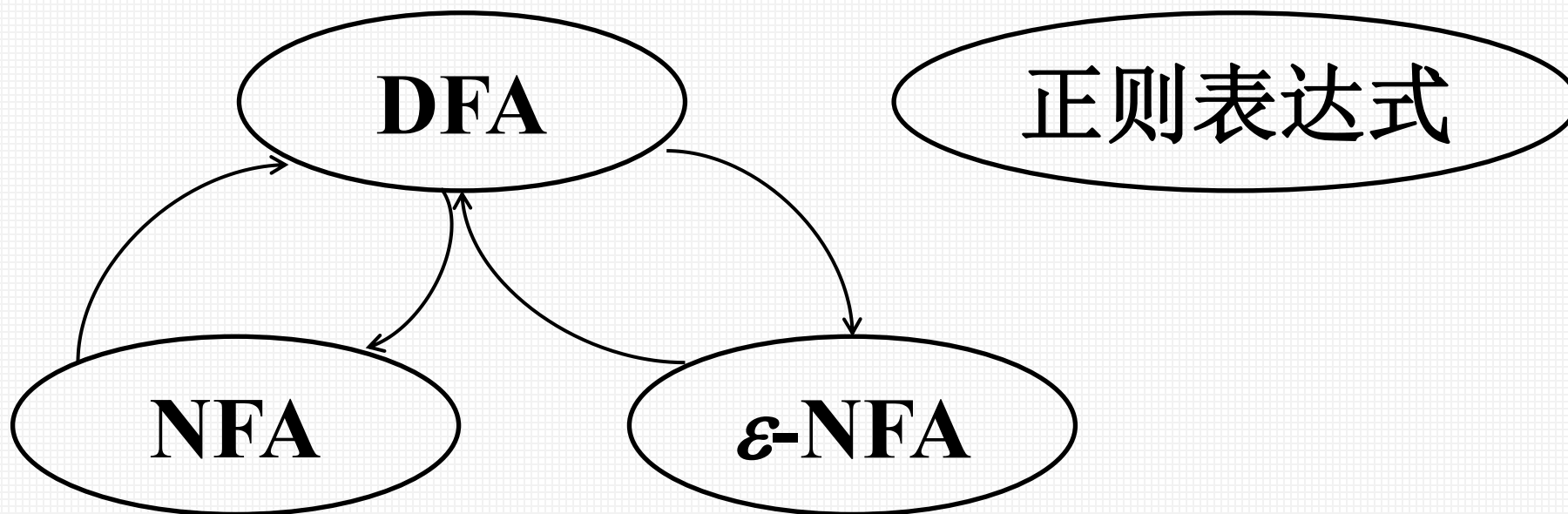
- $bb \rightarrow \{b\}\{b\} = \{bb\}, (a + bb) = \{a, bb\}$

- $(a + b)^*(a + bb) = \{a, b\}^*\{a, bb\} = \{a, bb, aa, abb, ba, bbb, \dots\}$

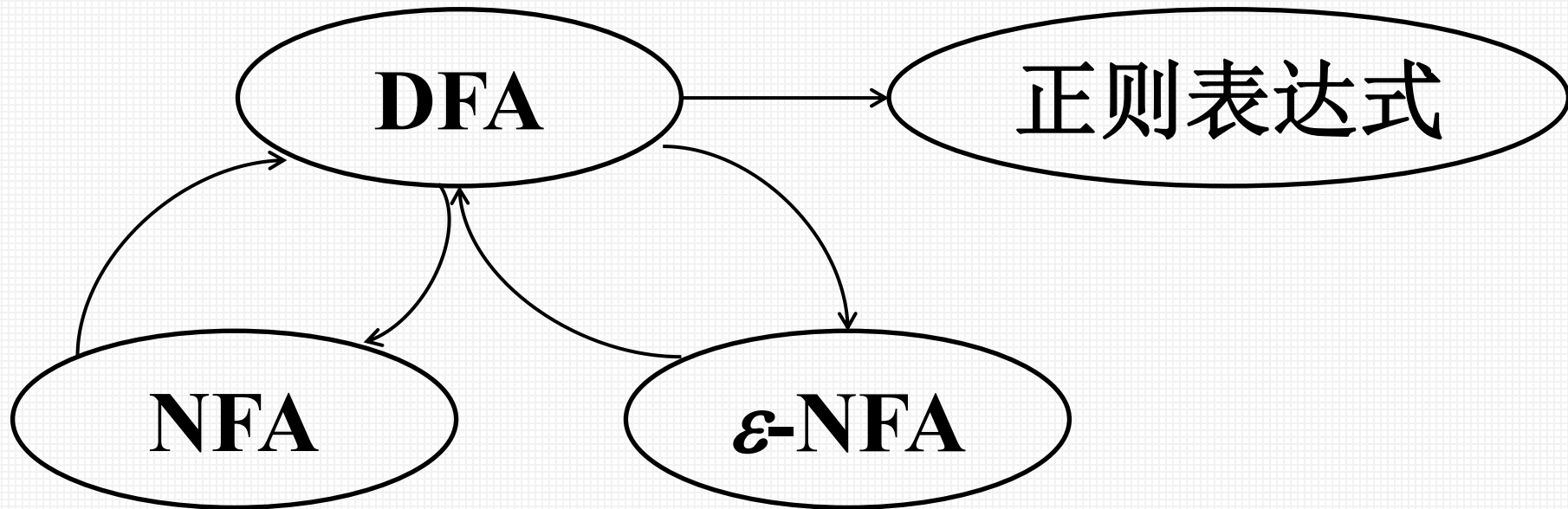
所以 $L((a + b)^*(a + bb)) = \{w | w \text{ 由 } a \text{ 和 } b \text{ 组成, 仅以 } a \text{ 或 } bb \text{ 结尾}\}$ 。

- 正则表达式
 - 正则表达式的递归定义
 - 正则表达式的示例
- **有穷自动机和正则表达式**
 - 由自动机到正则表达式
 - 由正则表达式到自动机
- 正则表达式的代数定律

- DFA, NFA, ϵ -NFA和正则表达式在表示语言的能力上是等价的
 - 相互之间的转换关系

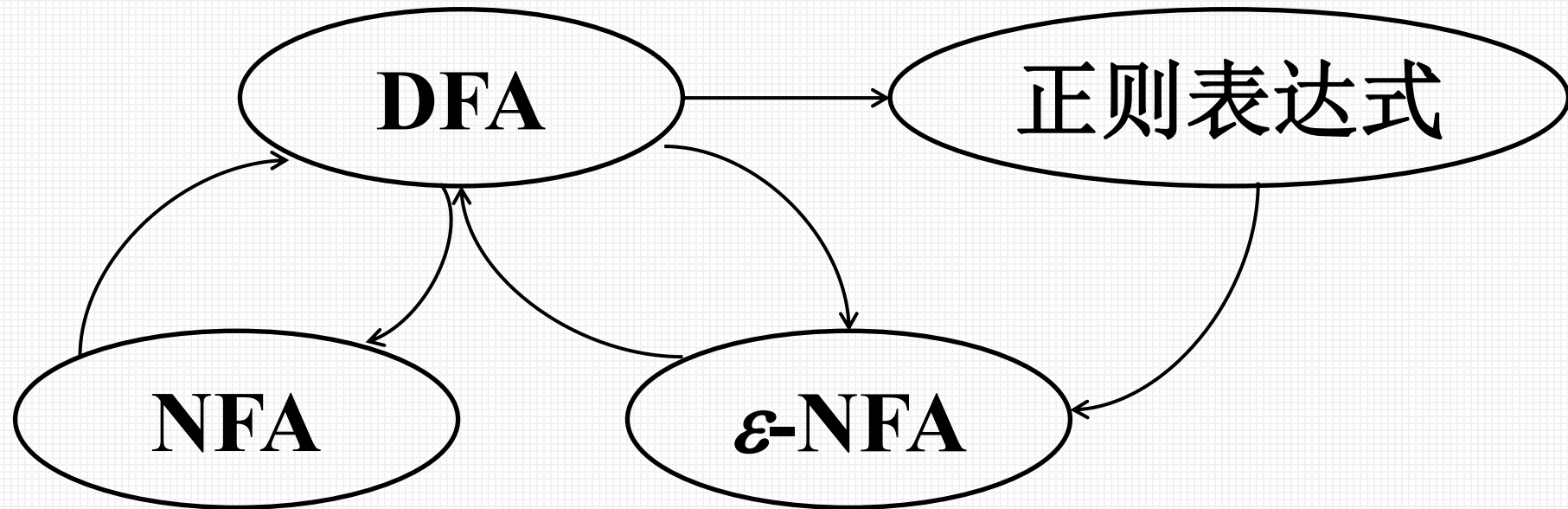


- **DFA, NFA, ϵ -NFA和正则表达式在表示语言的能力上是等价的**
 - 相互之间的转换关系



对于一个DFA所能识别的语言，都存在一个正则表达式表示它。

- DFA, NFA, ε -NFA和正则表达式在表示语言的能力上是等价的
 - 相互之间的转换关系



任何一个正则表达式表示的语言，都可以由一个 ε -NFA识别。

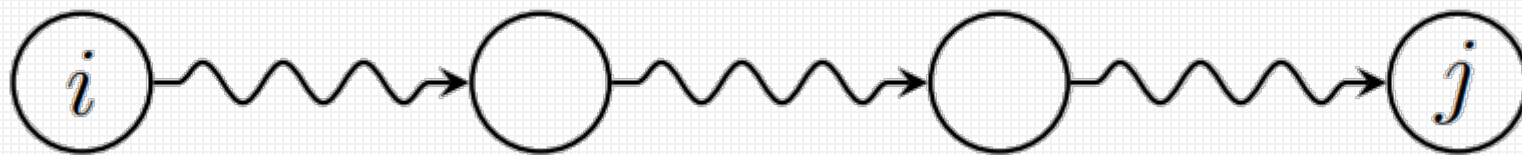
- **定理：**若 $L = L(A)$ 是某 DFA A 的语言，那么存在正则表达式 R 满足 $L = L(R)$ 。
- **证明方法：**
 - 递归法
 - 状态消除法

- 对 A 的状态进行编号, 令**1为开始状态**, 即

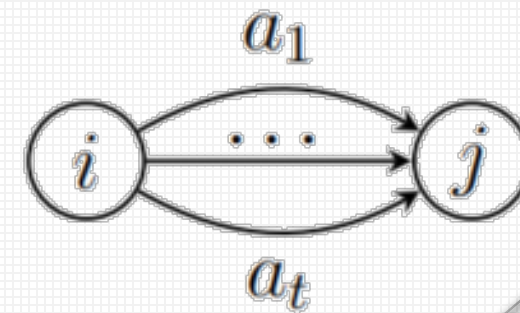
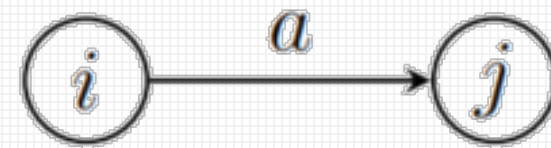
$$A = (\{1, 2, 3, \dots, n\}, \Sigma, \delta, 1, F)$$

设正则表达式 $R_{i,j}^{(k)}$ 表示从 i 到 j 但**中间**节点状态编号不超过 k 全部路径的字符串集 (简称 k -路径):

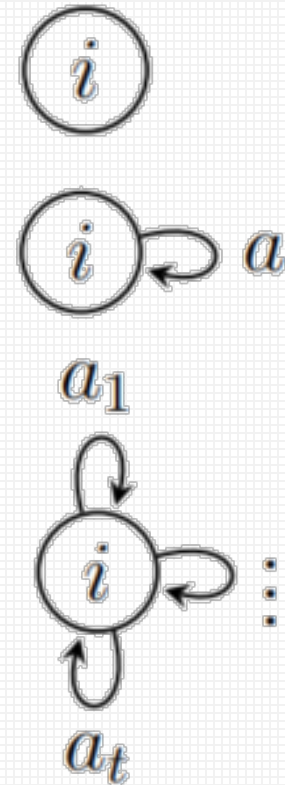
$$R_{i,j}^{(k)} = \{x \mid \hat{\delta}(i, x) = j, x \text{ 经过的状态除两端外都不超过 } k\}$$



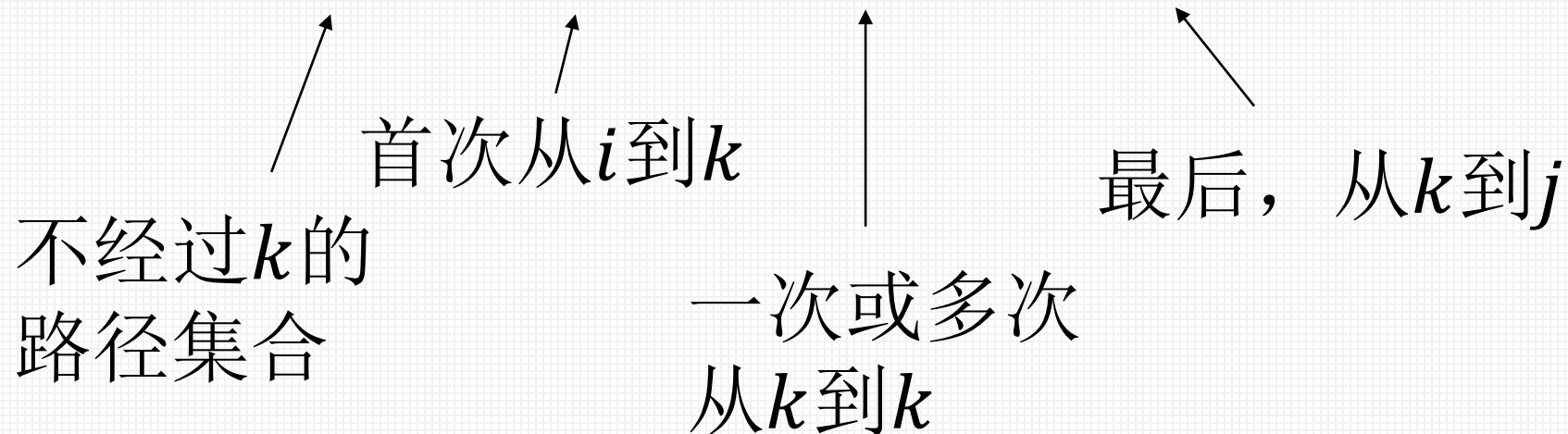
- 归纳基础: $k = 0$, $R_{i,j}^{(0)}$ 为从 i 直接到 j , 没有经过任何中间节点的字符串集合
- 情况1: i 不等于 j
 - 没有 i 到 j 的状态转移 $R_{i,j}^{(0)} = \emptyset$
 - 有一个 i 到 j 的转移 $R_{i,j}^{(0)} = a$
 - 有多个 i 到 j 的转移 $R_{i,j}^{(0)} = a_1 + \dots + a_t$



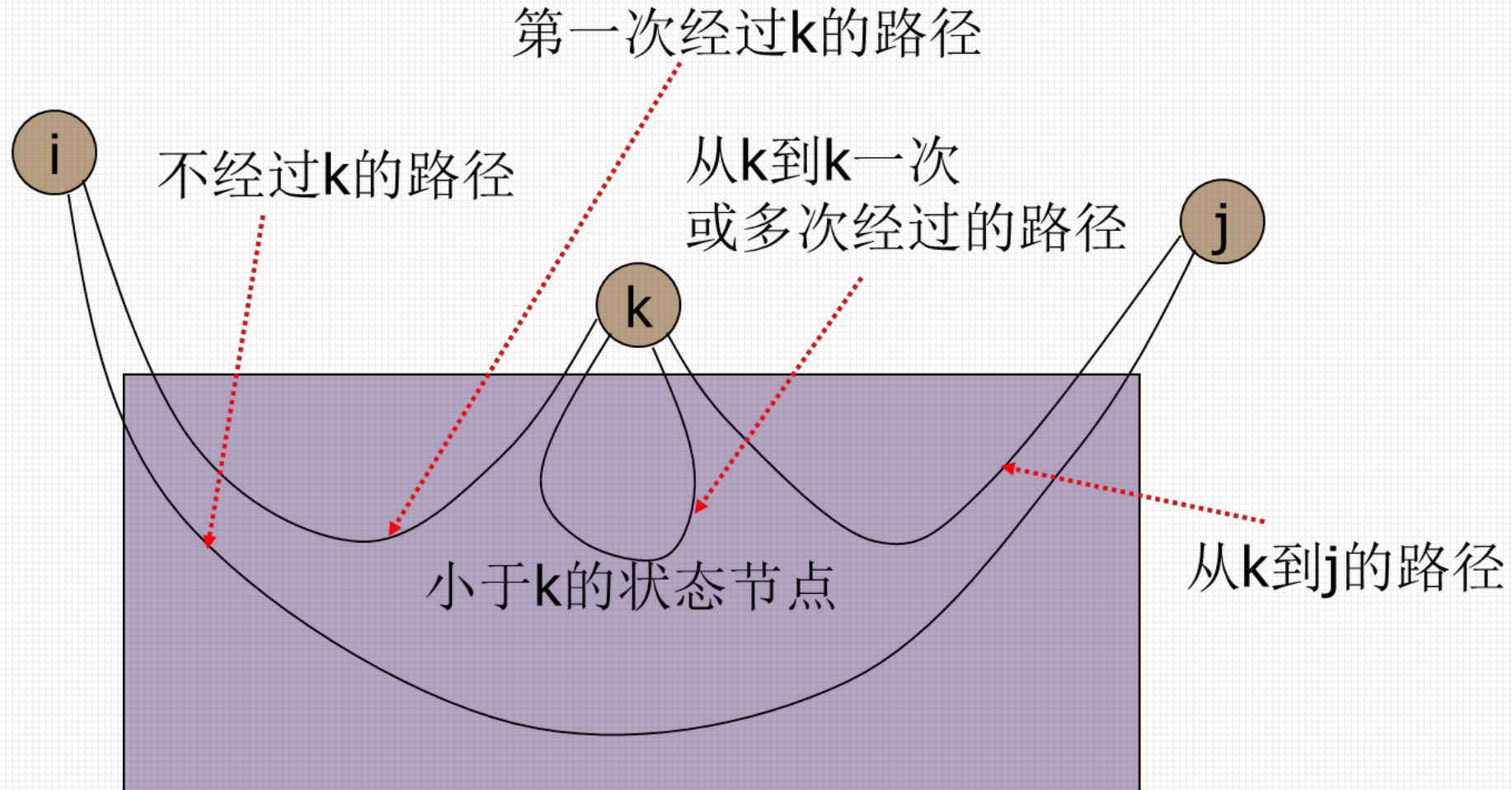
- 归纳基础: $k = 0$, $R_{i,j}^{(0)}$ 为从 i 直接到 j , 没有经过任何中间节点的字符串集合
- 情况2: i 等于 j
 - 状态 i 没有到自身的转移 $R_{i,i}^{(0)} = \varepsilon$
 - 状态 i 有一个到自身的转移 $R_{i,i}^{(0)} = a + \varepsilon$
 - 状态 i 有多个到自身的转移 $R_{i,i}^{(0)} = a_1 + \dots + a_t + \varepsilon$



- $k > 0$, $R_{i,j}^{(k)}$ 含有两种情况:
 - 不经过状态 k , 此时所经过的状态的最大编号为 $k - 1$
 - 经过状态 k 一次或者多次
- $$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$



- 递归关系: $R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$

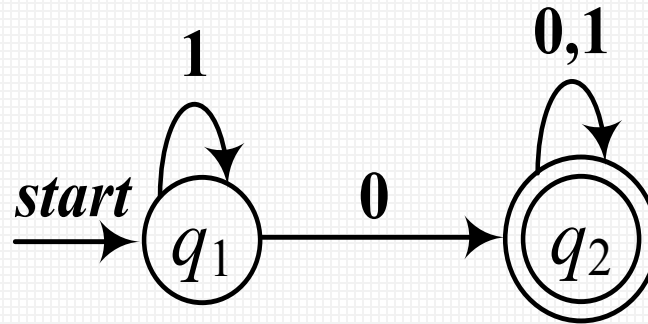


- 如果1是开始结点，则和DFA等价的正则表达式就是

$$\bigcup_{j \in F} R_{1,j}^{(n)}$$

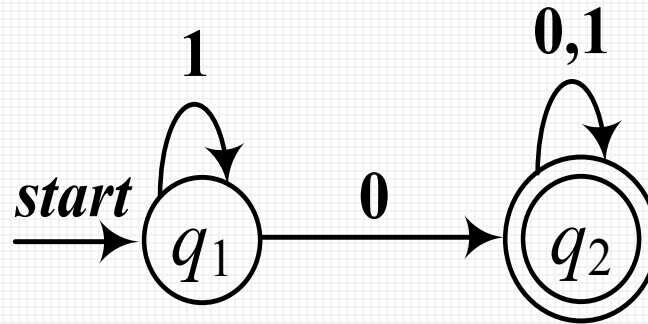
即，从状态1到终结状态的所有路径的集合。

- 例:把下图所示DFA转换为正则表达式。这个DFA 接受至少含有一个0的串。



- 状态总数 $n = 2$ ，状态1为开始状态，状态2为结束状态，需要求解？

- 例:把下图所示DFA转换为正则表达式。这个DFA 接受至少含有一个0的串。

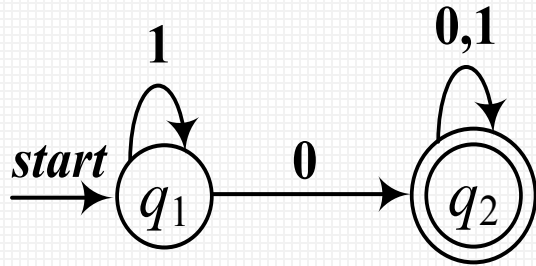


- 状态总数 $n = 2$ ，状态1为开始状态，状态2为结束状态，需要求解 $R_{1,2}^{(2)}$

- $(\varepsilon + E)^* = E^*$
- $E_1 + E_1 E_2^* = E_1 E_2^*$
- $\emptyset E_1 = E_1 \emptyset = \emptyset$ (零元)
- $\emptyset + E_1 = E_1 + \emptyset = E_1$ (单位元)

DFA→正则表达式示例

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

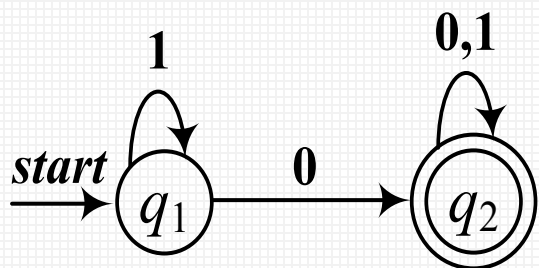
$k = 0$ 的情况
(状态*i*直接
到状态*j*):

$R_{1,1}^{(0)}$	
$R_{1,2}^{(0)}$	
$R_{2,1}^{(0)}$	
$R_{2,2}^{(0)}$	

$R_{1,1}^{(1)}$	
$R_{1,2}^{(1)}$	
$R_{2,1}^{(1)}$	
$R_{2,2}^{(1)}$	

$R_{1,1}^{(2)}$	
$R_{1,2}^{(2)}$	
$R_{2,1}^{(2)}$	
$R_{2,2}^{(2)}$	

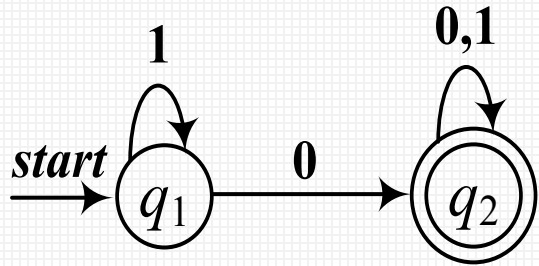
- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$k = 0$ 的情况
(状态 i 直接
到状态 j):

$R_{1,1}^{(0)}$	
$R_{1,2}^{(0)}$	
$R_{2,1}^{(0)}$	
$R_{2,2}^{(0)}$	

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$k = 0$ 的情况
(状态 i 直接
到状态 j):

$R_{1,1}^{(0)}$	$\varepsilon + 1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon + \mathbf{0} + 1$

情况1: i 不等于 j

没有 i 到 j 的状态转移 $R_{i,j}^{(0)} = \emptyset$

有一个 i 到 j 的转移 $R_{i,j}^{(0)} = \mathbf{a}$

有多个 i 到 j 的转移 $R_{i,j}^{(0)} = \mathbf{a}_1 + \cdots + \mathbf{a}_t$

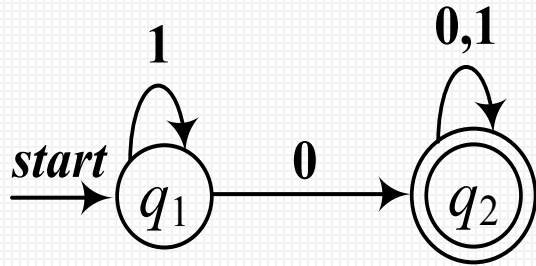
情况2: i 等于 j

状态 i 没有到自身的转移 $R_{i,j}^{(0)} = \varepsilon$

状态 i 有一个到自身的转移 $R_{i,j}^{(0)} = \mathbf{a} + \varepsilon$

状态 i 有多个到自身的转移 $R_{i,j}^{(0)} = \mathbf{a}_1 + \cdots + \mathbf{a}_t + \varepsilon$

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

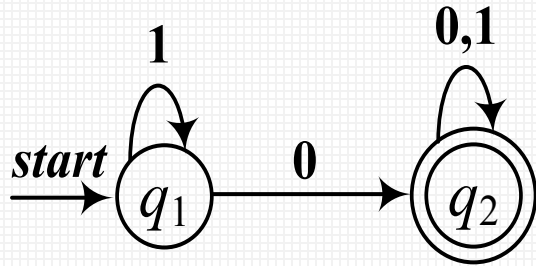
$k = 0$ 的情况
(状态*i*直接
到状态*j*):

$R_{1,1}^{(0)}$	$\varepsilon + 1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon + \mathbf{0} + 1$

$R_{1,1}^{(1)}$	$\varepsilon + 1 + (\varepsilon + 1)(\varepsilon + 1)^*(\varepsilon + 1)$
$R_{1,2}^{(1)}$	$\mathbf{0} + (\varepsilon + 1)(\varepsilon + 1)^*\mathbf{0}$
$R_{2,1}^{(1)}$	$\emptyset + \emptyset(\varepsilon + 1)^*(\varepsilon + 1)$
$R_{2,2}^{(1)}$	$\varepsilon + \mathbf{0} + 1 + \emptyset(\varepsilon + 1)^*\mathbf{0}$

DFA→正则表达式示例

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

$k = 0$ 的情况
(状态*i*直接
到状态*j*):

$R_{1,1}^{(0)}$	$\varepsilon + 1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon + \mathbf{0} + 1$

$R_{1,1}^{(1)}$	$\varepsilon + 1 + (\varepsilon + 1)(\varepsilon + 1)^*(\varepsilon + 1) = 1^*$
$R_{1,2}^{(1)}$	$\mathbf{0} + (\varepsilon + 1)(\varepsilon + 1)^*\mathbf{0} = 1^*\mathbf{0}$
$R_{2,1}^{(1)}$	$\emptyset + \emptyset(\varepsilon + 1)^*(\varepsilon + 1) = \emptyset$
$R_{2,2}^{(1)}$	$\varepsilon + \mathbf{0} + 1 + \emptyset(\varepsilon + 1)^*\mathbf{0} = \varepsilon + \mathbf{0} + 1$

简化规则:

$$(\varepsilon + E)^* = E^*$$

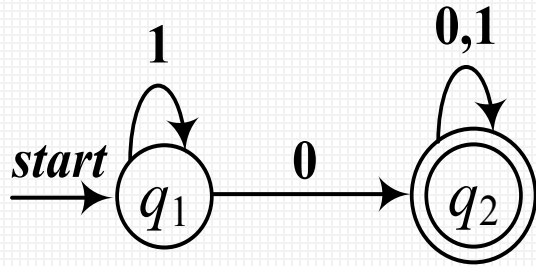
$$E_1 + E_1 E_2^* = E_1 E_2^*$$

$$\emptyset E_1 = E_1 \emptyset = \emptyset$$

$$\emptyset + E_1 = E_1 + \emptyset = E_1$$

DFA→正则表达式示例

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

$R_{1,1}^{(1)}$	1^*
$R_{1,2}^{(1)}$	1^*0
$R_{2,1}^{(1)}$	$\emptyset$
$R_{2,2}^{(1)}$	$\varepsilon + 0 + 1$

$k = 0$ 的情况
(状态*i*直接
到状态*j*):

$R_{1,1}^{(0)}$	$\varepsilon + 1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon + 0 + 1$

简化规则:

$$(\varepsilon + E)^* = E^*$$

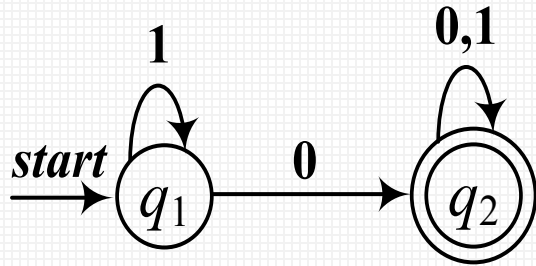
$$E_1 + E_1 E_2^* = E_1 E_2^*$$

$$\emptyset E_1 = E_1 \emptyset = \emptyset$$

$$\emptyset + E_1 = E_1 + \emptyset = E_1$$

DFA→正则表达式示例

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

$R_{1,1}^{(1)}$	1^*
$R_{1,2}^{(1)}$	1^*0
$R_{2,1}^{(1)}$	$\emptyset$
$R_{2,2}^{(1)}$	$\varepsilon + 0 + 1$

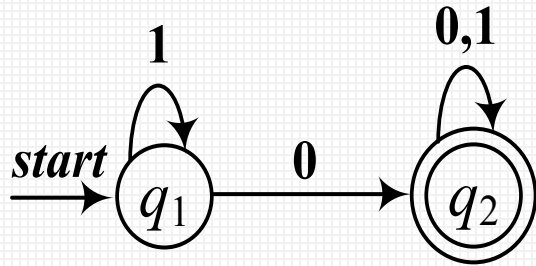
$k = 0$ 的情况
(状态*i*直接
到状态*j*):

$R_{1,1}^{(0)}$	$\varepsilon + 1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon + 0 + 1$

$R_{1,1}^{(2)}$	$1^* + 1^*0(\varepsilon + 0 + 1)^*\emptyset$
$R_{1,2}^{(2)}$	$1^*0 + 1^*0(\varepsilon + 0 + 1)^*(\varepsilon + 0 + 1)$
$R_{2,1}^{(2)}$	$\emptyset + (\varepsilon + 0 + 1)(\varepsilon + 0 + 1)^*\emptyset$
$R_{2,2}^{(2)}$	$\varepsilon + 0 + 1 + (\varepsilon + 0 + 1)(\varepsilon + 0 + 1)^*(\varepsilon + 0 + 1)$

DFA→正则表达式示例

- 把下图所示DFA转换为正则表达式。这个DFA接受至少含有一个0的串。



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

$R_{1,1}^{(1)}$	1^*
$R_{1,2}^{(1)}$	1^*0
$R_{2,1}^{(1)}$	$\emptyset$
$R_{2,2}^{(1)}$	$\varepsilon + 0 + 1$

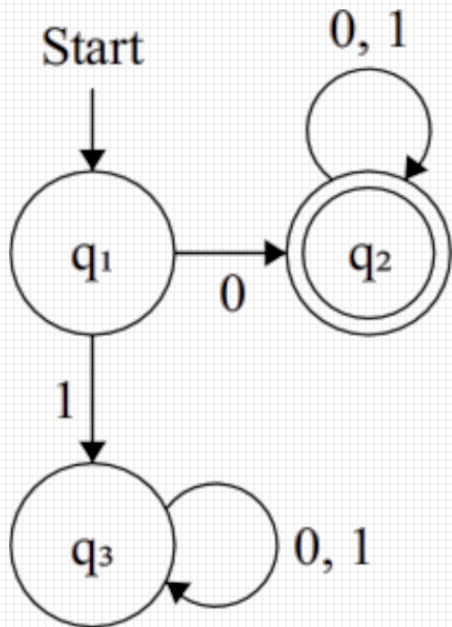
$k = 0$ 的情况
(状态*i*直接
到状态*j*):

$R_{1,1}^{(0)}$	$\varepsilon + 1$
$R_{1,2}^{(0)}$	0
$R_{2,1}^{(0)}$	$\emptyset$
$R_{2,2}^{(0)}$	$\varepsilon + 0 + 1$

$R_{1,1}^{(2)}$	1^*
$R_{1,2}^{(2)}$	$1^*0(0+1)^*$
$R_{2,1}^{(2)}$	$\emptyset$
$R_{2,2}^{(2)}$	$(0+1)^*$

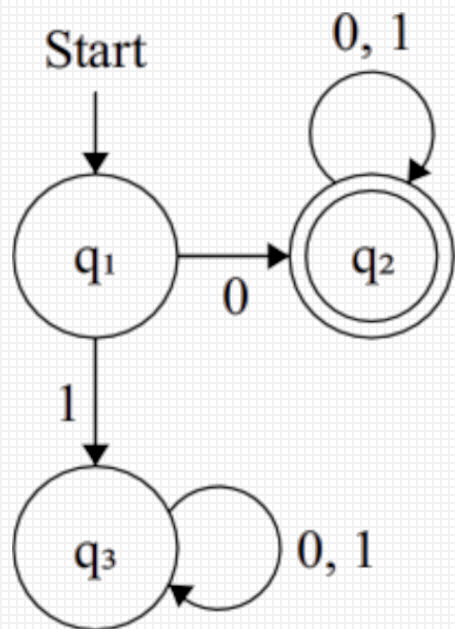
$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

- 将以下DFA转化成正则表达式：



$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

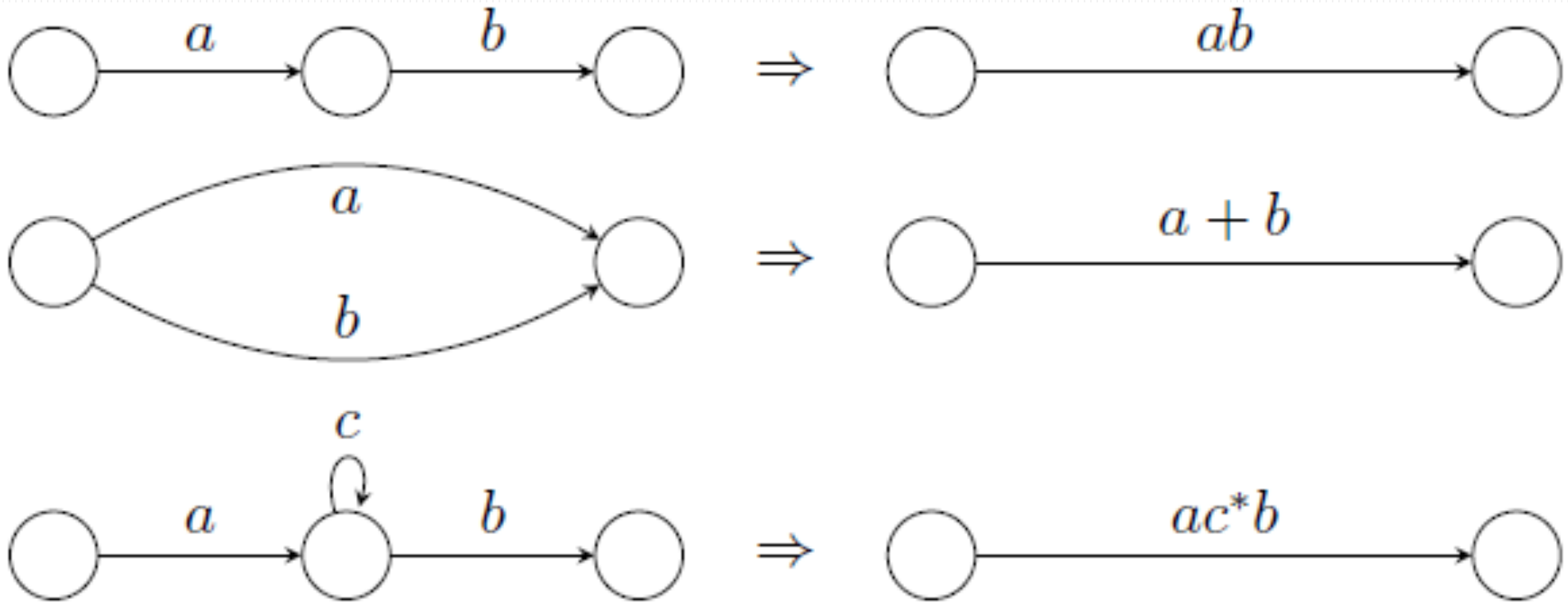
- 将以下DFA转化成正则表达式：



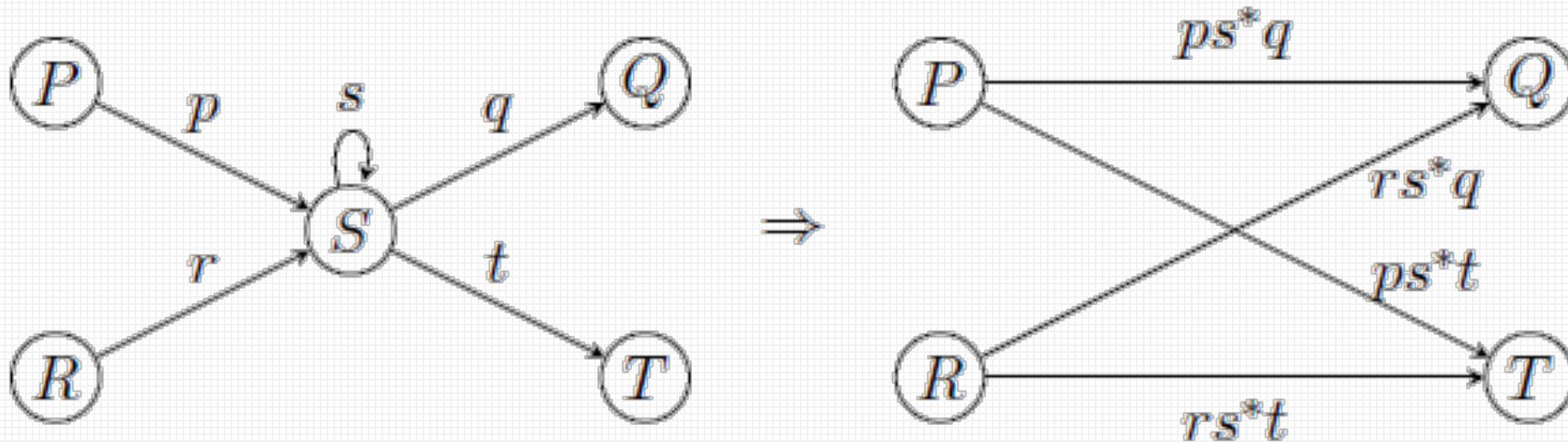
	$k = 0$	$k = 1$	$k = 2$	$k = 3$
R_{11}^k	ε	ε	ε	ε
R_{12}^k	0	0	0(0 + 1)*	0(0 + 1)*
R_{13}^k	1	1	1	1(0 + 1)*
R_{21}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{22}^k	0 + 1 + ε	0 + 1 + ε	(0 + 1)*	(0 + 1)*
R_{23}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{31}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{32}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{33}^k	0+1+ε	0+1+ε	0+1+ε	(0 + 1)*

- **从DFA中逐个删除状态**
 - 会消除掉状态的原有路径，因此要补上等价的路径
- **用标记了正则表达式的新路径替换被删掉的路径**
- **保持“自动机”等价**

• 正则表达式运算（连接，加，闭包）和DFA中路径的关系



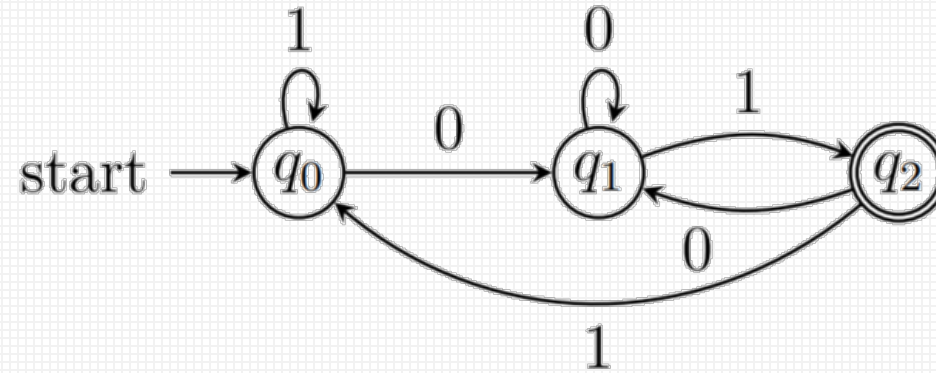
- 更一般的情况，要为被删除的状态 S 的每个“入”和“出”路径的组合，补一条等价的新路径，并用新的正则表达式表示



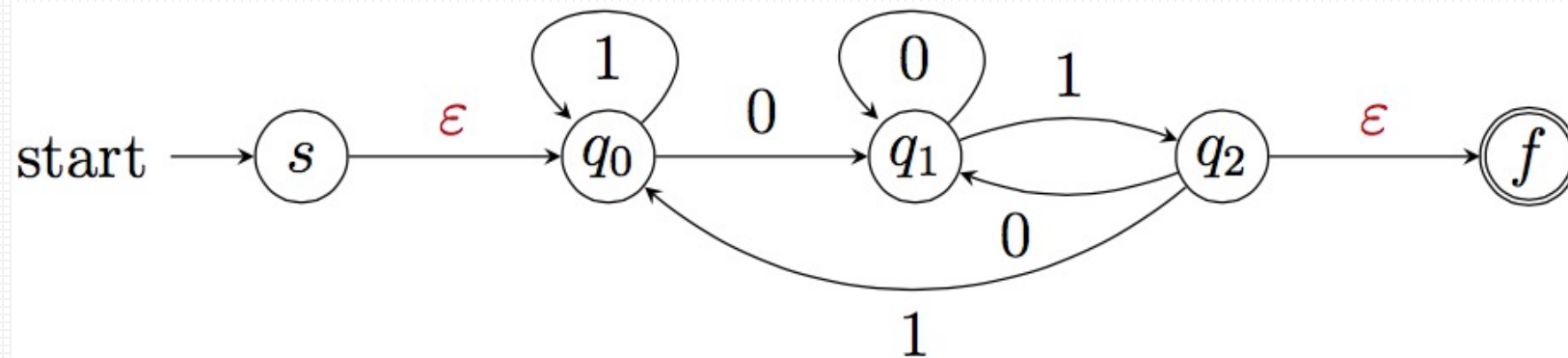
- 2条入，2条出，要补4条（ P 到 Q ， P 到 T ， R 到 Q ， R 到 T ）
 - 循环的回路另外考虑

- 例：由0和1构成的串中，接受全部以01结尾的串。

DFA:

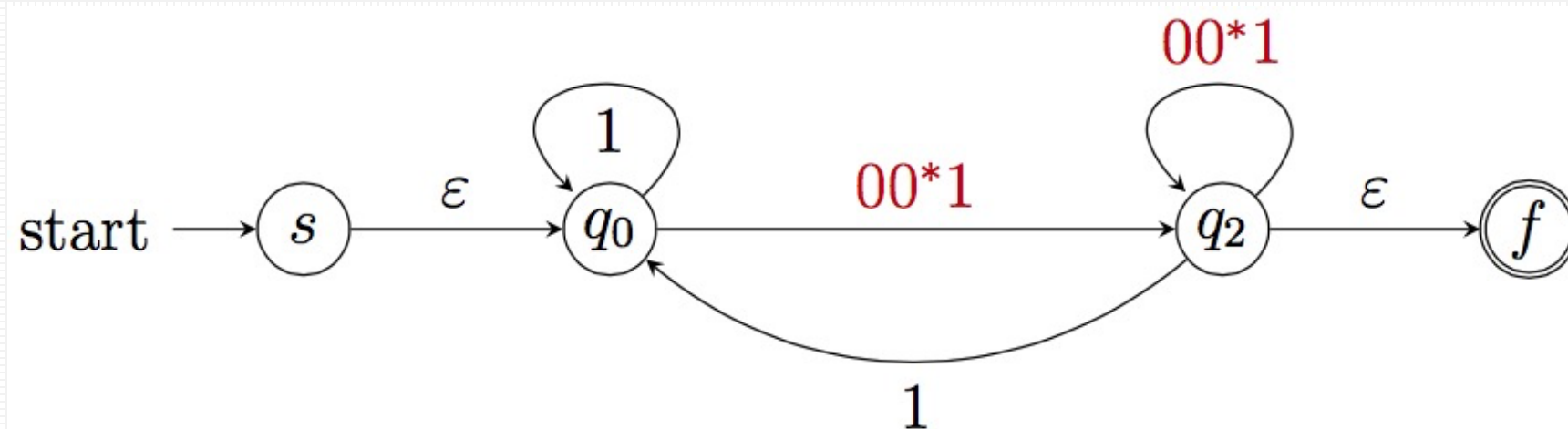
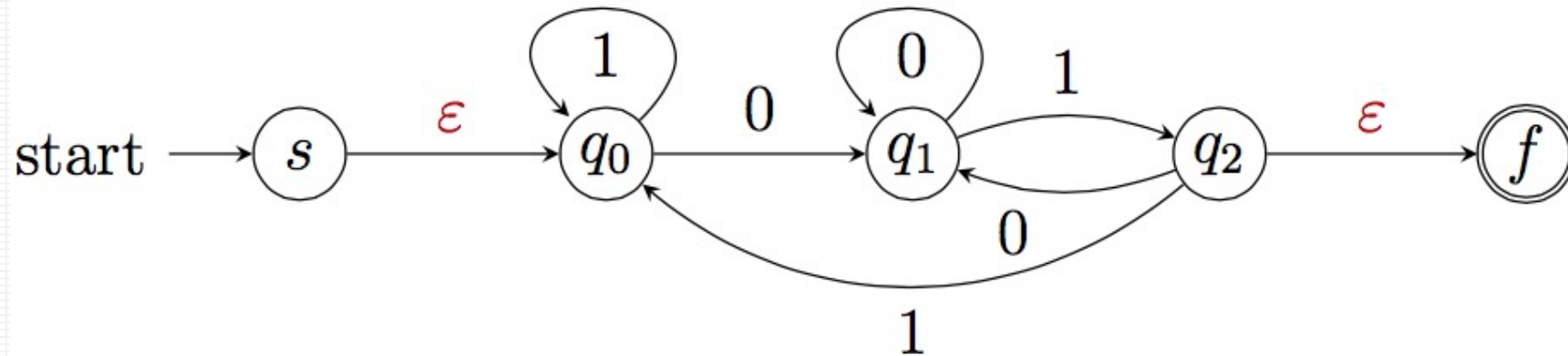


(1) 利用空转移，增加新的开始 (s) 和结束状态 (f)



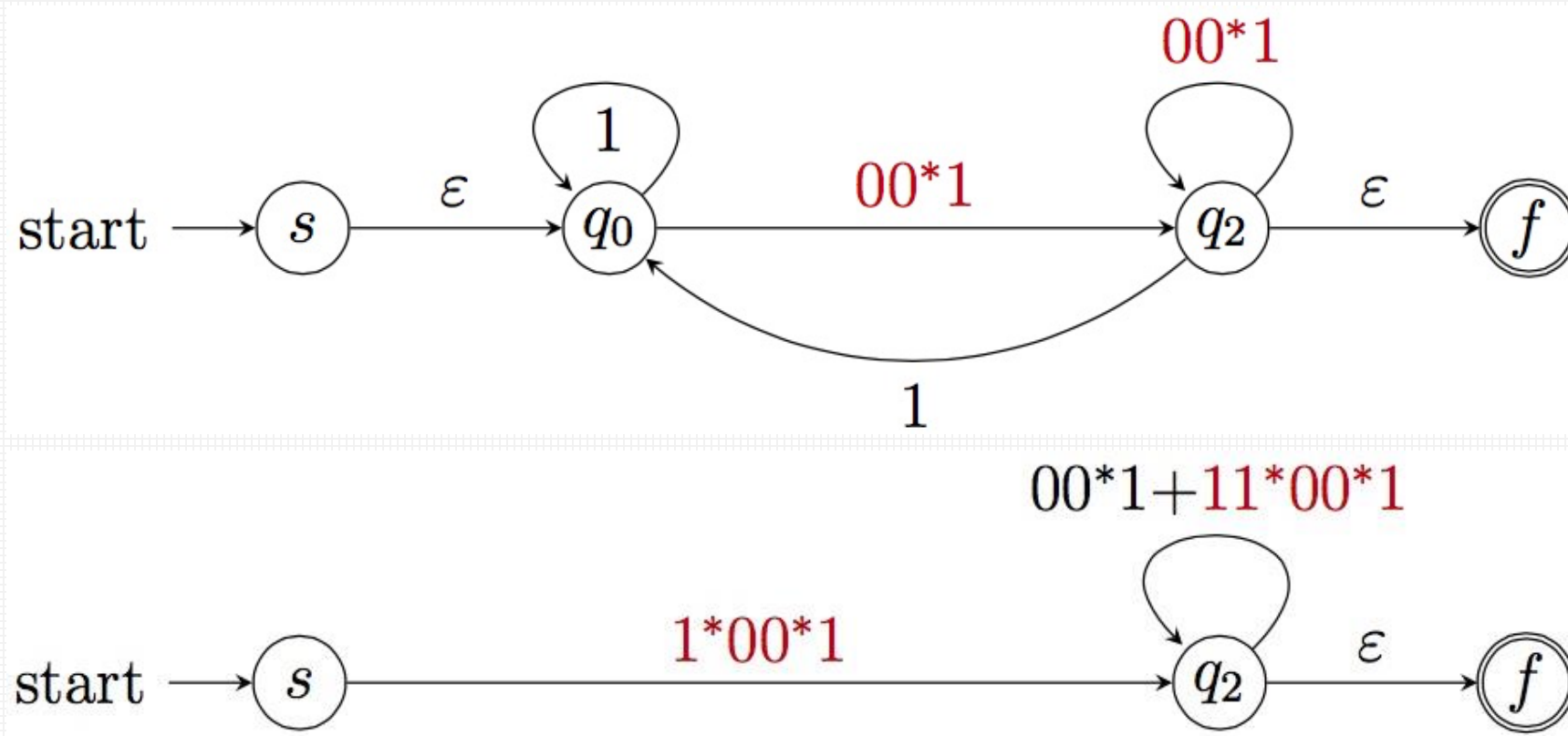
状态消除法示例

(2) 消除状态 q_1 ，需要增加其“入”度 $\{q_0, q_2\}$ 和“出”度 $\{q_2\}$ 之间的边 $q_0 00^*1q_2$ 和 $q_2 00^*1q_2$ ，即 $q_0 \rightarrow q_2$ 和 $q_2 \rightarrow q_2$

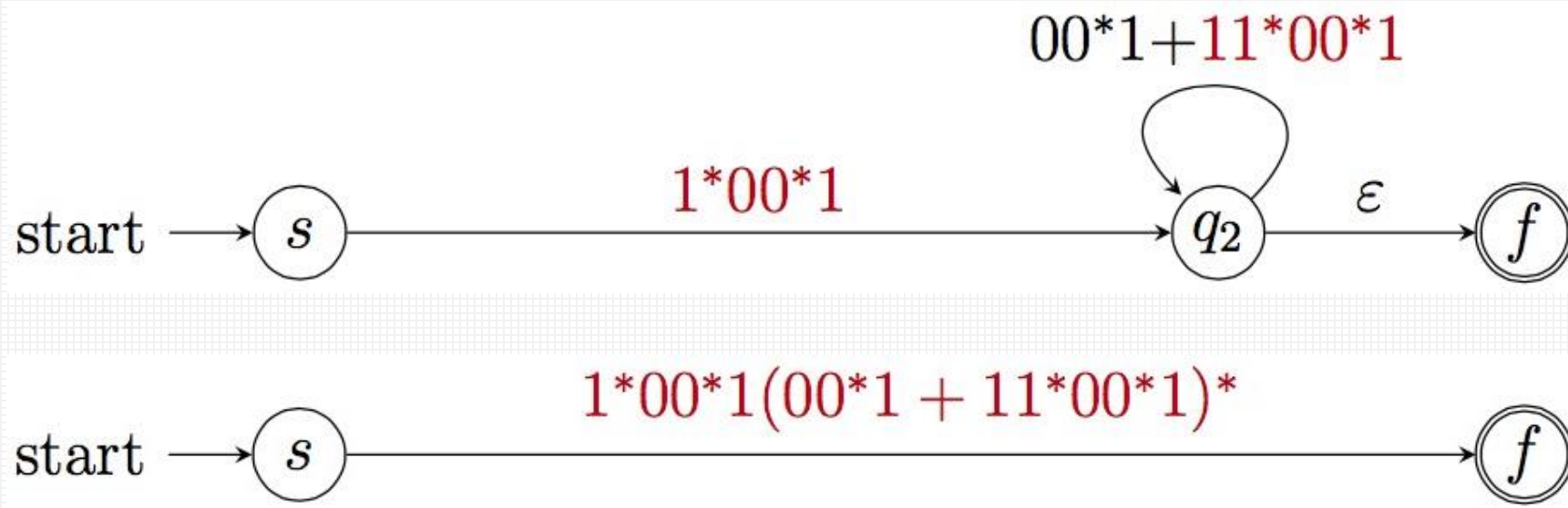


状态消除法示例

(3) 消除状态 q_0 ，需要增加其“入”度 $\{s, q_2\}$ 和“出”度 $\{q_2\}$ 之间的边 $s1^*00^*1q_2$ 和 $q_211^*00^*1q_2$ ，即 $s \rightarrow q_2$ 和 $q_2 \rightarrow q_2$



(4) 消除状态 q_2 , 需要增加其“入”度 $\{s\}$ 和“出”度 $\{f\}$ 之间的边 $s1^*00^*1(00^*1 + 11^*00^*1)^*f$, 即 $s \rightarrow f$

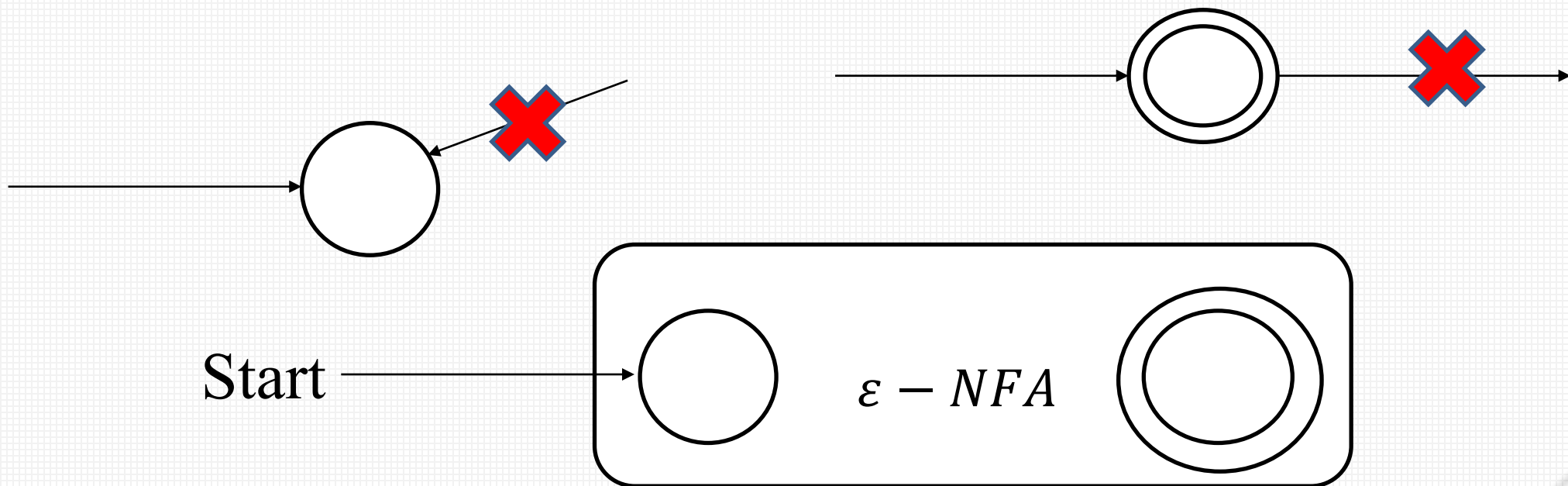


因此, 该自动机的正则表达式为:

$$1^*00^*1(00^*1 + 11^*00^*1)^*$$

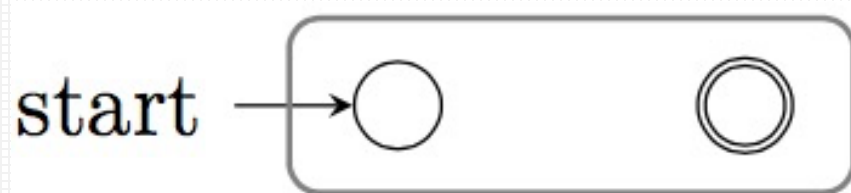
- 正则表达式
 - 正则表达式的递归定义
 - 正则表达式的示例
- **有穷自动机和正则表达式**
 - 由自动机到正则表达式
 - 由正则表达式到自动机
- 正则表达式的代数定律

- 定理: 每个正则表达式定义的语言, 都可被有穷自动机识别。
- $\Leftrightarrow$ 任何正则表达式 R , 都存在与其等价的 ε -NFA A , 即 $L(A) = L(R)$, 并且 A 满足:
 - 仅有一个接受状态;
 - 没有进入开始状态的边;
 - 没有离开接受状态的边。

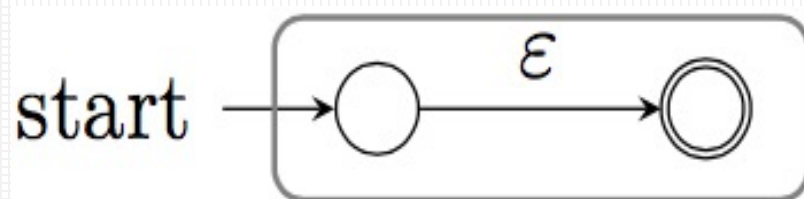


- 归纳基础:

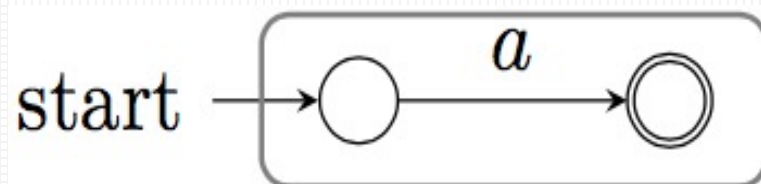
1. 对正则表达式 $\emptyset$, 有 ε -NFA



2. 对正则表达式 ε , 有 ε -NFA

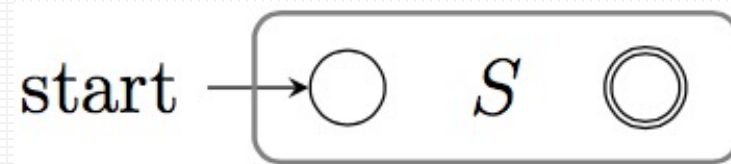
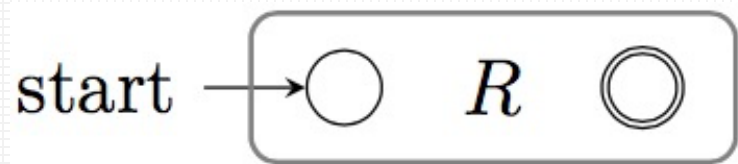


3. $\forall a \in \Sigma$, 对正则表达式 a , 有 ε -NFA



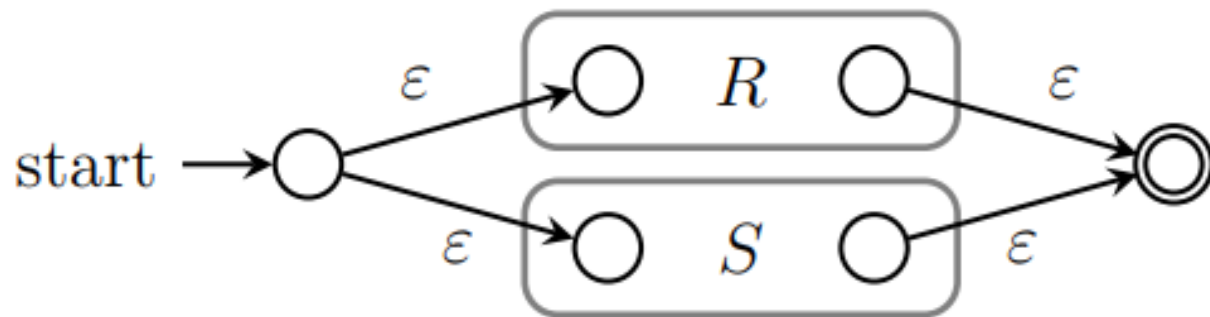
- 归纳递推:

假设若 r 和 s 为正则表达式, 则它们对应的 ε -NFA分别为 R 和 S 。



- 归纳递推:

$r + s$ 的 ε -NFA:



- 添加一个开始状态和一个接受状态；
- 将 R 和 S 通过空转移以并的方式连接起来。

- 归纳递推:

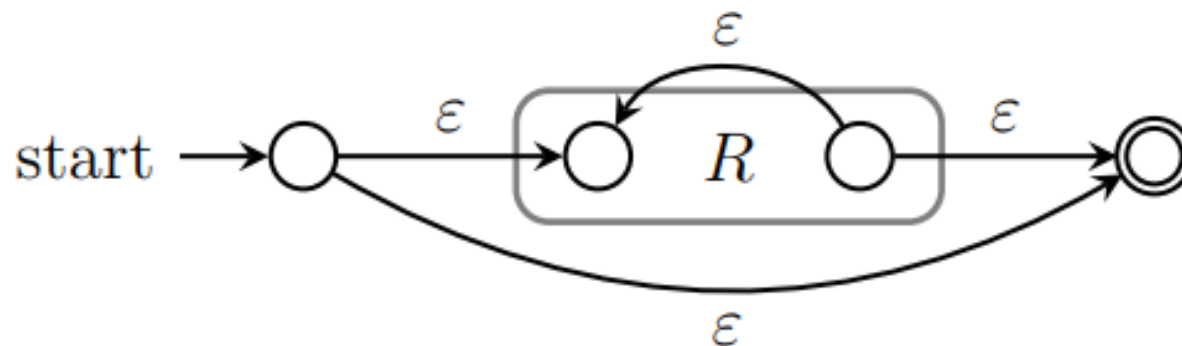
rs 的 ε -NFA:



- 通过空转移将 R 的接受状态和 S 的开始状态连接起来。

• 归纳递推:

r^* 的 ε -NFA:

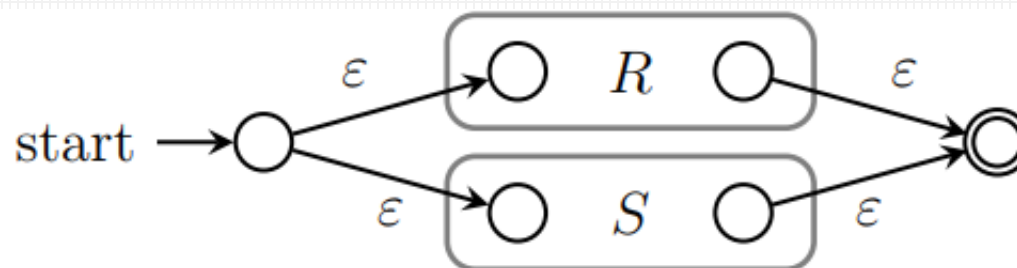


- 添加一个开始状态和一个接受状态；
- 通过空转移将开始状态连接，将接受状态连接；
- 把原来的接受状态改为非接受状态，通过空转移跳转回原来的开始状态（表示闭包里的 >0 次）；
- 闭包里的0次通过空转移连接开始状态和接受状态实现。

- 归纳递推:

若 r 和 s 为正则表达式, 则它们对应的 ε -NFA别为 R 和 S 。

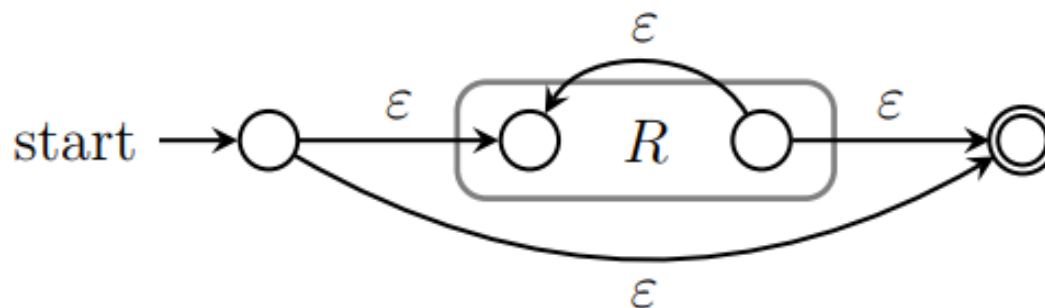
$r + s$ 的 ε -NFA:



rs 的 ε -NFA:

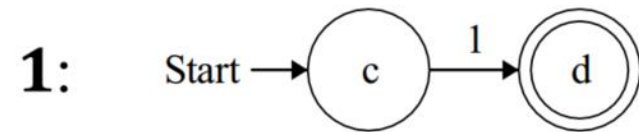
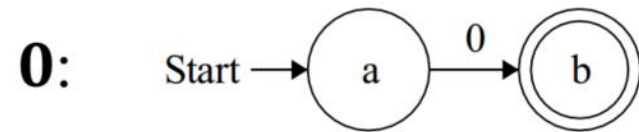


r^* 的 ε -NFA:

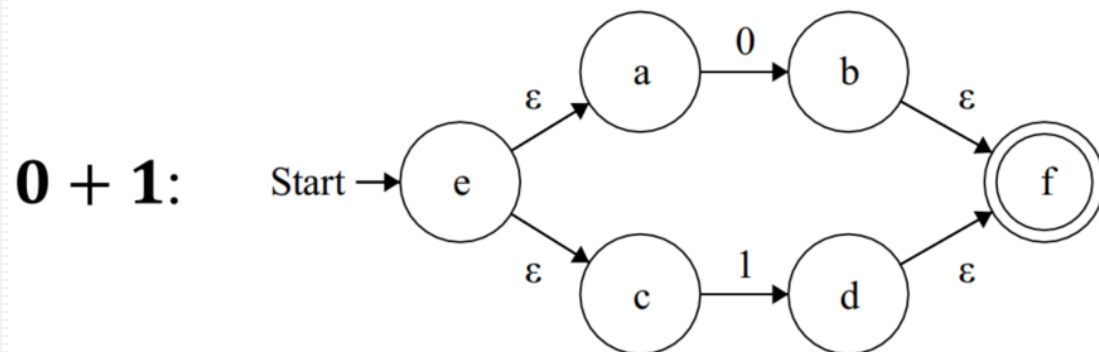
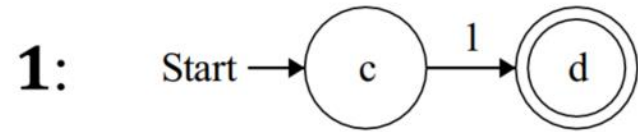
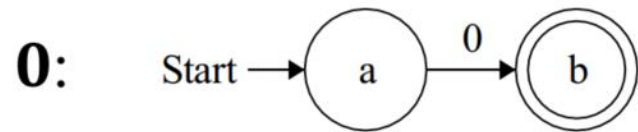


因此对于任何正则表达式, 可以构造等价的 ε -NFA。

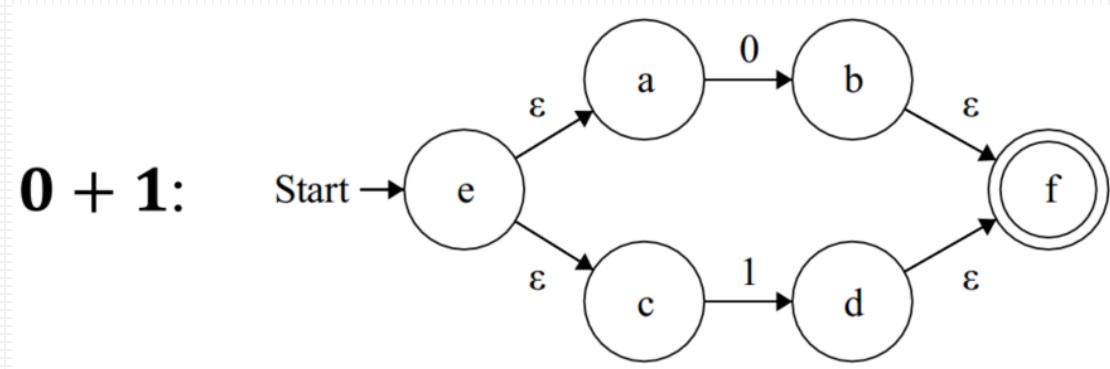
- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



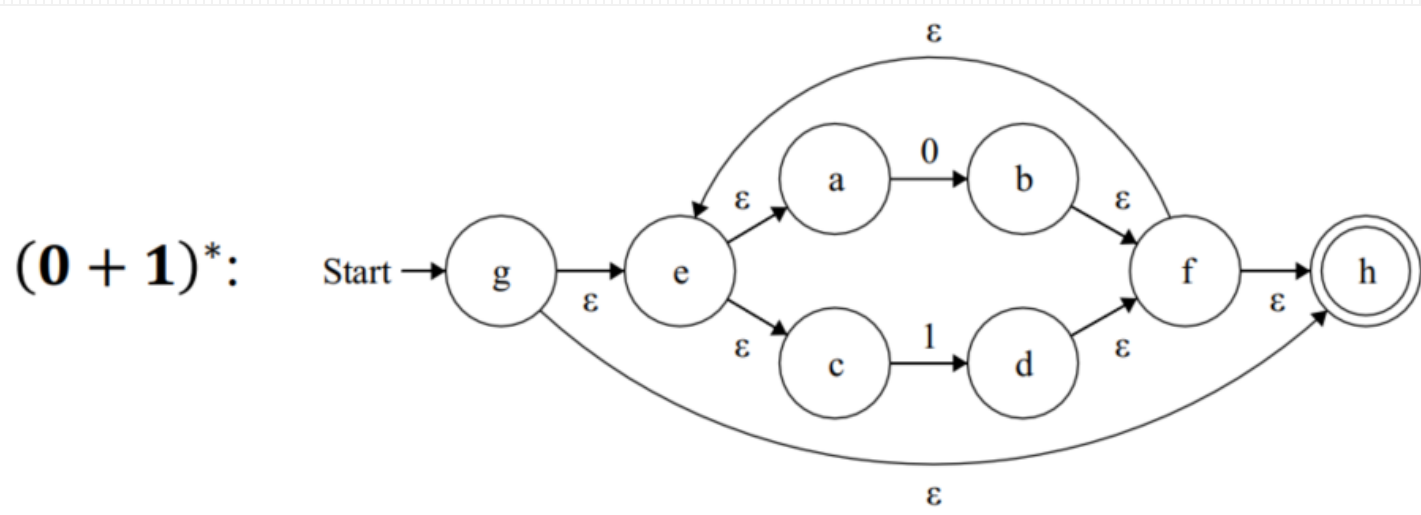
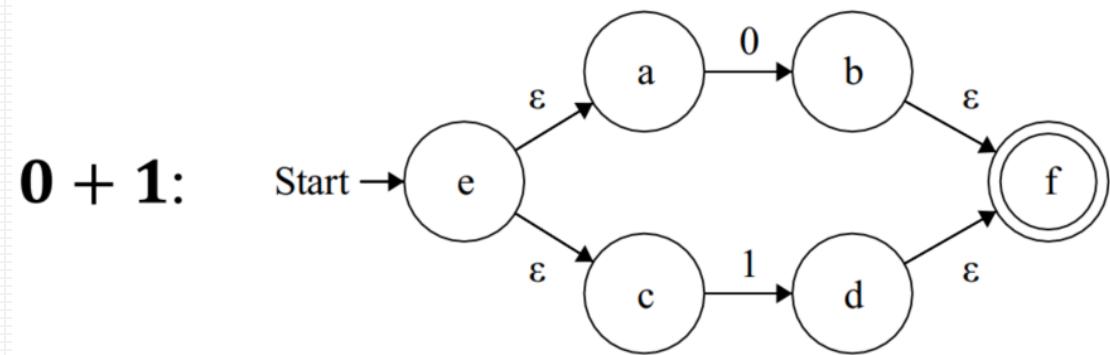
- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



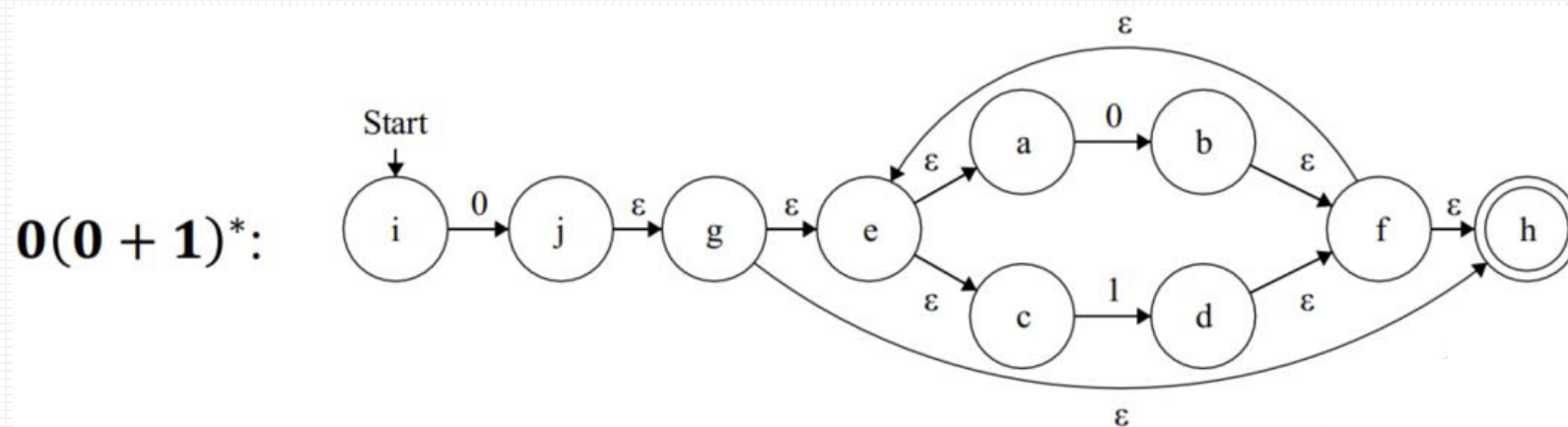
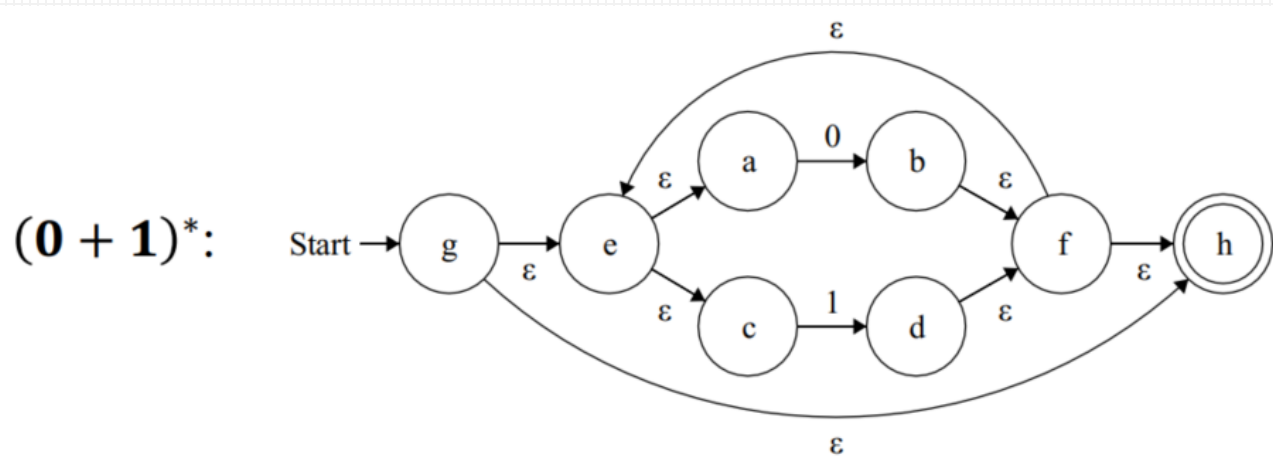
- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



- **正则表达式**
 - 正则表达式的递归定义
 - 正则表达式的示例
- **有穷自动机和正则表达式**
 - 由自动机到正则表达式
 - 由正则表达式到自动机
- **正则表达式的代数定律**

- 结合律和交换律

- $L + M = M + L$ 并的交换律
- $(L + M) + N = L + (M + N)$ 并的结合律
- $(LM)N = L(MN)$ 连接的结合律
- 连接不满足交换律 $LM \neq ML$

- 单位元

- $\emptyset + L = L + \emptyset = L$ 并运算的单位元 $\emptyset$
- $\varepsilon L = L\varepsilon = L$ 连接运算的单位元 ε

- 零元

- $\emptyset L = L\emptyset = \emptyset$ 连接运算的零元

- 分配律

- $L(M + N) = LM + LN$

- $(M + N)L = ML + NL$

示例: $0+01^*=0\varepsilon+01^*=0(\varepsilon+1^*)=01^*$

- 幂等律

- $L+L=L$

- 有关闭包的定律 (Laws involving closures)
 - $(L^*)^* = L^*$, 对某语言的闭包再取闭包
 - $\emptyset^* = \varepsilon$
 - $\varepsilon^* = \varepsilon$ 空串正则表达式的闭包仍然是本身
 - $LL^* = L^*L$
 - $(L + \varepsilon)^* = L^*$

- 正则表达式基本概念
- 正则表达式与有穷自动机的转换
 - DFA $\rightarrow$ 正则表达式：递归法 + 状态消除法
 - 正则表达式 $\rightarrow$ DFA
- 正则表达式的代数定律